Name : Vasita Zeel M.
Roll No : 27

Dataset:-1.

Write a Python program for the following data

(a) Make scatter plot. (b) calculate the linear regression line for the data using the model:

f□□□ = □0 +□1 ×□□□□□□□□

(i). Calculate the regression coefficients using the formula.

(ii). Find Parameters:

◇ What is the slope □1?

◇ What is the y-intercept □0?

◇ Provide the units of each.

(iii). Draw the Regression Line

(c) Calculate SSE for the line fitted.

(d) Plot SSE graph □(□0) by varying □0 while keeping □1 fixed as in the fitted

equation.

(e) Plot SSE graph □(□1) by varying □1while keeping □0fixed in the fitted equation.

(f) Plot □(□0,□1).

(g) Calculate the linear regression equation using Gradient Descent Method.

(h) Use the matrix equation to derive the parameters:

B =(□□□)−1□□□

(i) Calculate the regression line using Python libraries and verify the results obtained in (g), (b) - i, and (h).

(j) Compute Metrics □2, MSE, MAE, RMSE.

Q.1 The following table reports the distance (in miles) from Baltimore to each of

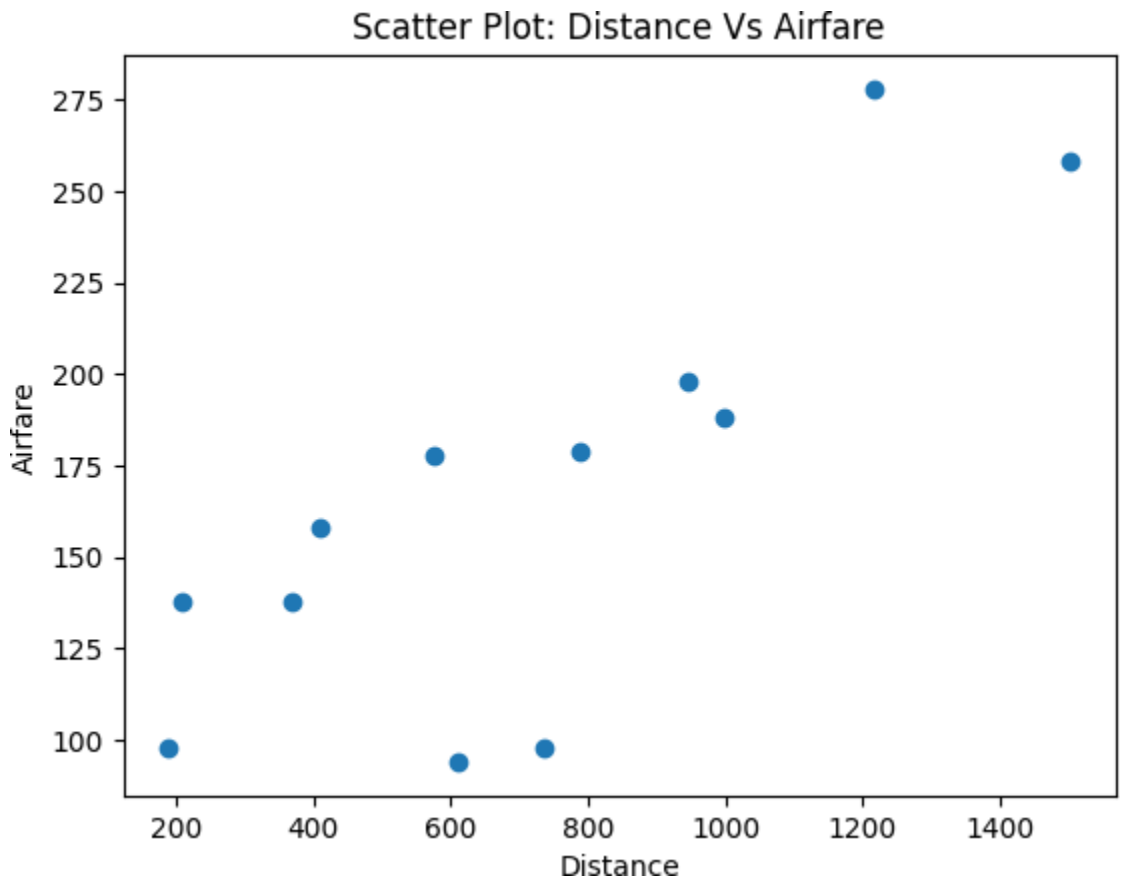
the 12 cities along with the corresponding airfare

Destination	Distance (miles)	Airfare ($)
Atlanta	576	178
Boston	370	138
Chicago	612	94
Dallas	1216	278
Detroit	409	158
Denver	1502	258
Miami	946	198
New Orleans	998	188
New York	189	98
Orlando	787	179
Pittsburgh	210	138
St. Louis	737	98

(a) Make scatter plot.

In [80]:
```
import matplotlib.pyplot as plt

x = [576, 370, 612, 1216, 409, 1502, 946, 998, 189, 787, 210, 737]
y  = [178, 138, 94, 278, 158, 258, 198, 188, 98, 179, 138, 98]
```

In [81]:
```
plt.scatter(x,y)
plt.xlabel("Distance")
plt.ylabel("Airfare")

plt.title("Scatter Plot: Distance Vs Airfare")
plt.show()
```

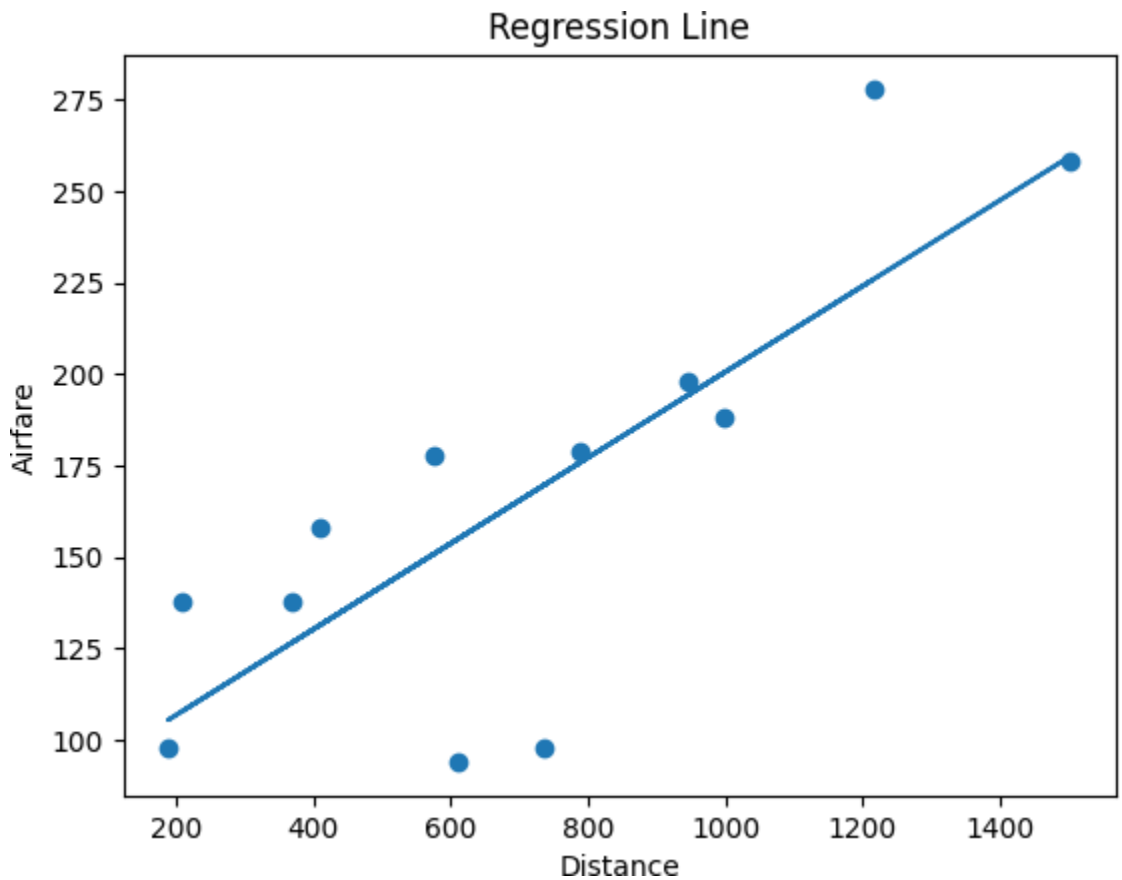


In [82]:

```
n = len(x)

sumX = 0
sumY = 0
sumXY = 0
sumX2 = 0

# calculat the sum using loop
for i in range(n):
    sumX += x[i]
    sumY += y[i]
    sumXY += x[i] * y[i]
    sumX2 += x[i] * x[i]

# table print

print("Table:")
print("X\tY\tXY\tX^2")
for i in range(n):
    xy = x[i] * y[i]
    x2 =  x[i] * x[i]
    print(x[i] , "\t" , y[i] , "\t" , xy , "\t" ,x2)

#Calculat the slop

a1 = (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX)
```

```python
#Calculat the intercept

a0 = (sumY * sumX2 - sumX * sumXY) /(n * sumX2 - sumX * sumX)

print("Slop(a1):" , a1)
print("Intercerpt(a0)",a0)

print("Equation:")
print(f"{a0} + {a1}x")
# Drew the Regression Line

plt.scatter(x,y)

y_pred = []
for i in range(n):
    y_valu = a0+a1 * x[i]
    y_pred.append(y_valu)

plt.plot(x,y_pred)
plt.xlabel("Distance")
plt.ylabel("Airfare")
plt.title("Regression Line")

plt.show()
```

```
Table:
X         Y         XY         X^2
576           178           102528           331776
370           138           51060           136900
612           94           57528           374544
1216           278           338048           1478656
409           158           64622           167281
1502           258           387516           2256004
946           198           187308           894916
998           188           187624           996004
189           98           18522           35721
787           179           140873           619369
210           138           28980           44100
737           98           72226           543169
Slop(a1): 0.11737508839231386
Intercerpt(a0) 83.26735367241098
Equation:
83.26735367241098 + 0.11737508839231386x
```



Calculate SSE

In [83]:
```
sse = 0

print("\nx\ty\ty_pred\tError\terror^2")

for i in range(n):
    y_pred = a0 +a1 * x[i]
    error = y[i] - y_pred
    error_squer = error ** 2

    sse += error_squer

    print(x[i], "\t", y[i], "\t", round(y_pred,2), "\t", round(error,2), "\t",

print("SSE:" , sse)
```

```
x         y         y_pred         Error         error^2
576           178           150.88           27.12           735.74
370           138           126.7           11.3           127.78
612           94           155.1           -61.1           3733.32
1216           278           226.0           52.0           2704.47
409           158           131.27           26.73           714.29
1502           258           259.56           -1.56           2.45
946           198           194.3           3.7           13.66
998           188           200.41           -12.41           153.95
189           98           105.45           -7.45           55.52
787           179           175.64           3.36           11.28
210           138           107.92           30.08           905.04
737           98           169.77           -71.77           5151.33
SSE: 14308.837785527729
```

SSE Vs a0

In [84]:

```
a1 = 0.117

#diffrent a0
a0_val = [60,70,75,80,83.27,85,90,100]

sse_val = []

for a0 in a0_val:
    sse = 0
    print("\na0:" , a0)

    print("x\t\y\ty_pred\terror\error^2")

    for i in range(n):
        y_pred = a0+a1*x[i]
        error = y[i] - y_pred
        error_square = error ** 2

        sse += error_square
        print(x[i] ,"\t" ,y[i] ,"\t", round(y_pred,2),"\t" ,round(error,2) ,"\
    print("SSE=" , round(sse,2))
    sse_val.append(sse)

plt.plot(a0_val,sse_val,marker = "*")
plt.xlabel("a0 values")
plt.ylabel("SSE")
plt.title("SSE vs a0")
plt.show()
```

a0: 60

x	\y	y_pred	error\error^2
576	178	127.39	50.61
370	138	103.29	34.71
612	94	131.6	-37.6
1216	278	202.27	75.73
409	158	107.85	50.15
1502	258	235.73	22.27
946	198	170.68	27.32
998	188	176.77	11.23
189	98	82.11	15.89
787	179	152.08	26.92
210	138	84.57	53.43
737	98	146.23	-48.23

SSE= 20955.66

a0: 70

x	\y	y_pred	error\error^2
576	178	137.39	40.61
370	138	113.29	24.71
612	94	141.6	-47.6
1216	278	212.27	65.73
409	158	117.85	40.15
1502	258	245.73	12.27
946	198	180.68	17.32
998	188	186.77	1.23
189	98	92.11	5.89
787	179	162.08	16.92
210	138	94.57	43.43
737	98	156.23	-58.23

SSE= 16507.34

a0: 75

x	\y	y_pred	error\error^2
576	178	142.39	35.61
370	138	118.29	19.71
612	94	146.6	-52.6
1216	278	217.27	60.73
409	158	122.85	35.15
1502	258	250.73	7.27
946	198	185.68	12.32
998	188	191.77	-3.77
189	98	97.11	0.89
787	179	167.08	11.92
210	138	99.57	38.43
737	98	161.23	-63.23

SSE= 15183.18

a0: 80

x	\y	y_pred	error\error^2
576	178	147.39	30.61
370	138	123.29	14.71
612	94	151.6	-57.6
1216	278	222.27	55.73

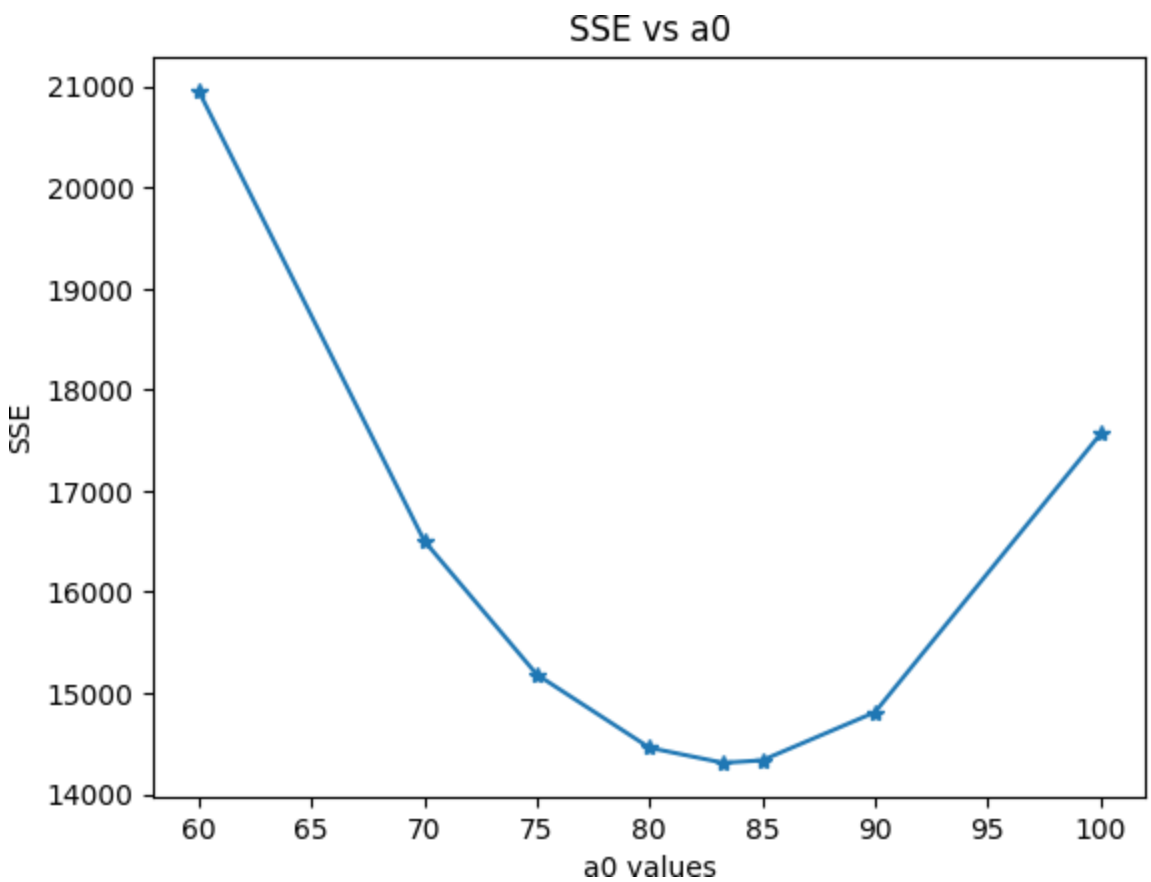
409 158 127.85 30.15
1502 258 255.73 2.27
946 198 190.68 7.32
998 188 196.77 -8.77
189 98 102.11 -4.11
787 179 172.08 6.92
210 138 104.57 33.43
737 98 166.23 -68.23
SSE= 14459.02

a0: 83.27
x \y y_pred error\error^2
576 178 150.66 27.34
370 138 126.56 11.44
612 94 154.87 -60.87
1216 278 225.54 52.46
409 158 131.12 26.88
1502 258 259.0 -1.0
946 198 193.95 4.05
998 188 200.04 -12.04
189 98 105.38 -7.38
787 179 175.35 3.65
210 138 107.84 30.16
737 98 169.5 -71.5
SSE= 14309.93

a0: 85
x \y y_pred error\error^2
576 178 152.39 25.61
370 138 128.29 9.71
612 94 156.6 -62.6
1216 278 227.27 50.73
409 158 132.85 25.15
1502 258 260.73 -2.73
946 198 195.68 2.32
998 188 201.77 -13.77
189 98 107.11 -9.11
787 179 177.08 1.92
210 138 109.57 28.43
737 98 171.23 -73.23
SSE= 14334.86

a0: 90
x \y y_pred error\error^2
576 178 157.39 20.61
370 138 133.29 4.71
612 94 161.6 -67.6
1216 278 232.27 45.73
409 158 137.85 20.15
1502 258 265.73 -7.73
946 198 200.68 -2.68
998 188 206.77 -18.77
189 98 112.11 -14.11
787 179 182.08 -3.08

```
210          138          114.57          23.43
737          98          176.23          -78.23
SSE= 14810.7

a0: 100
x        \y        y_pred        error\error^2
576          178          167.39          10.61
370          138          143.29          -5.29
612          94          171.6          -77.6
1216          278          242.27          35.73
409          158          147.85          10.15
1502          258          275.73          -17.73
946          198          210.68          -12.68
998          188          216.77          -28.77
189          98          122.11          -24.11
787          179          192.08          -13.08
210          138          124.57          13.43
737          98          186.23          -88.23
SSE= 17562.38
```



SSE Vs a1

In [85]:
```
a0 = 83.27

#diffrent a0
```

```
a1_val = [0.05, 0.08, 0.10, 0.117, 0.13, 0.15, 0.18]

sse_val = []

for a1 in a1_val:
    sse = 0
    print("\na1:" , a1)

    print("x\t\y\ty_pred\terror\error^2")

    for i in range(n):
        y_pred = a0+a1*x[i]
        error = y[i] - y_pred
        error_square = error ** 2

        sse += error_square
        print(x[i] ,"\t" ,y[i] ,"\t", round(y_pred,2),"\t" ,round(error,2) ,"\
    print("SSE=" , round(sse,2))
    sse_val.append(sse)

plt.plot(a1_val,sse_val,marker = "*")
plt.xlabel("a1 values")
plt.ylabel("SSE")
plt.title("SSE vs a1")
plt.show()
```

a1: 0.05

x	\y	y_pred	error\error^2
576	178	112.07	65.93
370	138	101.77	36.23
612	94	113.87	-19.87
1216	278	144.07	133.93
409	158	103.72	54.28
1502	258	158.37	99.63
946	198	130.57	67.43
998	188	133.17	54.83
189	98	92.72	5.28
787	179	122.62	56.38
210	138	93.77	44.23
737	98	120.12	-22.12

SSE= 50069.2

a1: 0.08

x	\y	y_pred	error\error^2
576	178	129.35	48.65
370	138	112.87	25.13
612	94	132.23	-38.23
1216	278	180.55	97.45
409	158	115.99	42.01
1502	258	203.43	54.57
946	198	158.95	39.05
998	188	163.11	24.89
189	98	98.39	-0.39
787	179	146.23	32.77
210	138	100.07	37.93
737	98	142.23	-44.23

SSE= 25312.52

a1: 0.1

x	\y	y_pred	error\error^2
576	178	140.87	37.13
370	138	120.27	17.73
612	94	144.47	-50.47
1216	278	204.87	73.13
409	158	124.17	33.83
1502	258	233.47	24.53
946	198	177.87	20.13
998	188	183.07	4.93
189	98	102.17	-4.17
787	179	161.97	17.03
210	138	104.27	33.73
737	98	156.97	-58.97

SSE= 16686.5

a1: 0.117

x	\y	y_pred	error\error^2
576	178	150.66	27.34
370	138	126.56	11.44
612	94	154.87	-60.87
1216	278	225.54	52.46

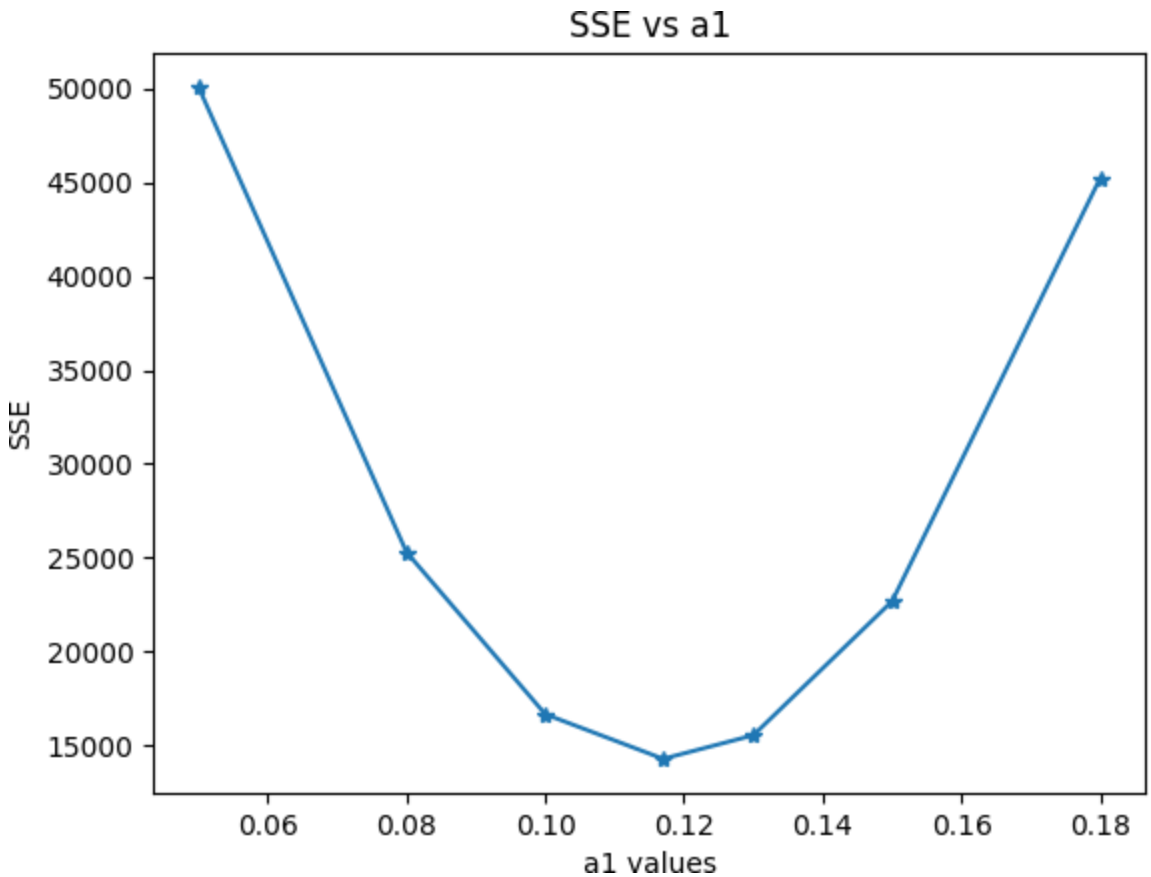
409 158 131.12 26.88
1502 258 259.0 -1.0
946 198 193.95 4.05
998 188 200.04 -12.04
189 98 105.38 -7.38
787 179 175.35 3.65
210 138 107.84 30.16
737 98 169.5 -71.5
SSE= 14309.93

a1: 0.13
x \y y_pred error\error^2
576 178 158.15 19.85
370 138 131.37 6.63
612 94 162.83 -68.83
1216 278 241.35 36.65
409 158 136.44 21.56
1502 258 278.53 -20.53
946 198 206.25 -8.25
998 188 213.01 -25.01
189 98 107.84 -9.84
787 179 185.58 -6.58
210 138 110.57 27.43
737 98 179.08 -81.08
SSE= 15565.14

a1: 0.15
x \y y_pred error\error^2
576 178 169.67 8.33
370 138 138.77 -0.77
612 94 175.07 -81.07
1216 278 265.67 12.33
409 158 144.62 13.38
1502 258 308.57 -50.57
946 198 225.17 -27.17
998 188 232.97 -44.97
189 98 111.62 -13.62
787 179 201.32 -22.32
210 138 114.77 23.23
737 98 193.82 -95.82
SSE= 22696.01

a1: 0.18
x \y y_pred error\error^2
576 178 186.95 -8.95
370 138 149.87 -11.87
612 94 193.43 -99.43
1216 278 302.15 -24.15
409 158 156.89 1.11
1502 258 353.63 -95.63
946 198 253.55 -55.55
998 188 262.91 -74.91
189 98 117.29 -19.29
787 179 224.93 -45.93

```
210            138            121.07            16.93
737            98            215.93           -117.93
SSE=  45209.97
```

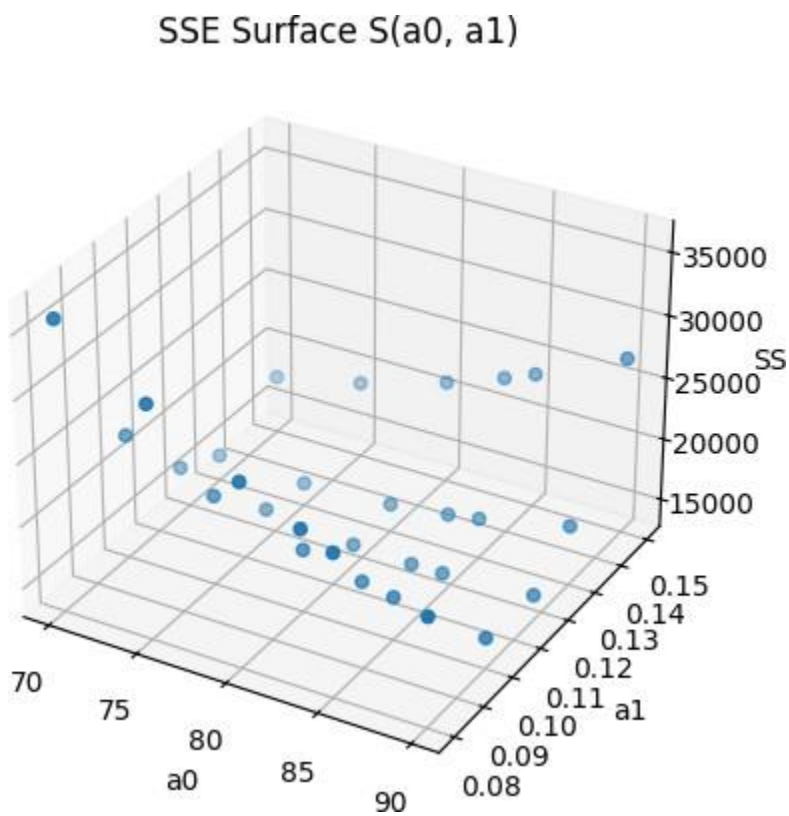


Surface area = s(a0,a1) Both value change

In [86]:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

a0_values = [70, 75, 80, 83.27, 85, 90]
a1_values = [0.08, 0.10, 0.117, 0.13, 0.15]

A0 = []
A1 = []
SSE = []

for a0 in a0_values:
    for a1 in a1_values:
        sse = 0

        for i in range(n):
            y_pred = a0 +a1 * x[i]
            error = y[i] - y_pred
            sse += error ** 2
```

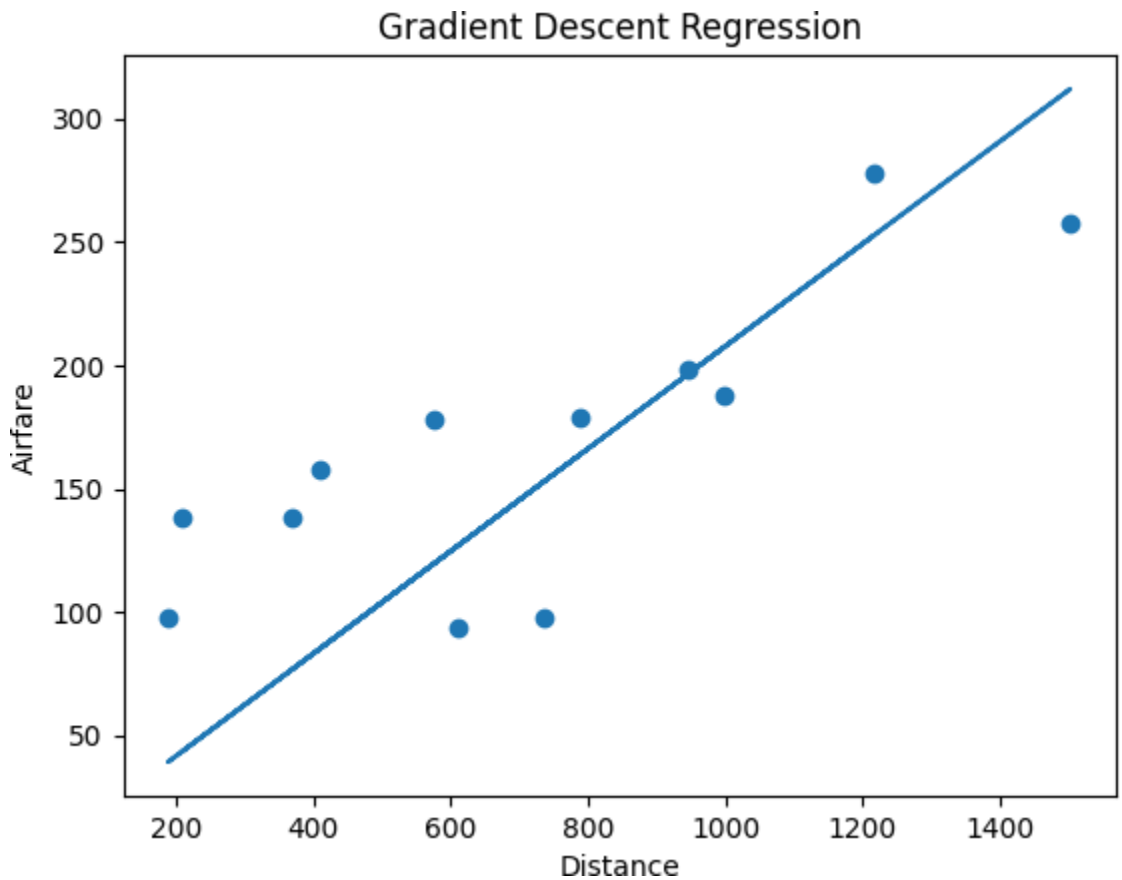
```
        A0.append(a0)
        A1.append(a1)
        SSE.append(sse)
#3d Surface

fig = plt.figure()
ax = fig.add_subplot(111,projection='3d')

ax.scatter(A0, A1, SSE)

ax.set_xlabel("a0")
ax.set_ylabel("a1")
ax.set_zlabel("SSE")

plt.title("SSE Surface S(a0, a1)")
plt.show()

#contor plot
# plt.tricontourf(A0, A1, SSE)
# plt.xlabel("a0")
# plt.ylabel("a1")
# plt.title("Contour Plot of SSE")
# plt.colorbar()
# plt.show()
```



g) Calculate the linear regression equation using Gradient Descent Method

In [91]:
```
a0 = 0
a1 = 0

learning_rate =0.0000001
e = 1000

for i in range(e):
    sum_err = 0
    sum_err_x = 0

    for j in range(n):
        y_pred = a0 + a1 * x[j]
        error = y[j] - y_pred
        sum_err += error
        sum_err_x += x[j] * error

    temp0 = a0 + (2/n) * learning_rate *sum_err
    temp1 = a1 + (2/n) * learning_rate * sum_err_x
    a0 =temp0
    a1 = temp1

print("a0:", round(a0,2))
print("a1:", round(a1,2))

plt.scatter(x, y)

y_pred = []
for i in range(n):
    y_val = a0 + a1 * x[i]
    y_pred.append(y_val)

plt.plot(x, y_pred)

plt.xlabel("Distance")
plt.ylabel("Airfare")
plt.title("Gradient Descent Regression")

plt.show()
```

```
a0: 0.0
a1: 0.21
```



In [ ]:
```
## Matricx
```

In [92]:
```
# Step 1: calculate sums
sumX = 0
sumY = 0
sumXY = 0
sumX2 = 0

for i in range(n):
    sumX += x[i]
    sumY += y[i]
    sumXY += x[i] * y[i]
    sumX2 += x[i] * x[i]

# Step 2: calculate coefficients using matrix formula

denominator = (n * sumX2 - sumX * sumX)

a1 = (n * sumXY - sumX * sumY) / denominator
a0 = (sumY * sumX2 - sumX * sumXY) / denominator

# Output
print("a0 (Intercept):", round(a0, 2))
print("a1 (Slope):", round(a1, 3))
```

```
a0 (Intercept): 83.27
a1 (Slope): 0.117
```

In [ ]:
```
## (i) Calculate the regression line using Python libraries and verify the res
obtained in (g), (b) - i, and (h).
```

In [93]:
```
from sklearn.linear_model import LinearRegression
import numpy as np
```

In [95]:
```
x = np.array([576, 370, 612, 1216, 409, 1502, 946, 998, 189, 787, 210, 737])
y = np.array([178, 138, 94, 278, 158, 258, 198, 188, 98, 179, 138, 98])

x = x.reshape(-1,1)

model = LinearRegression()
model.fit(x,y)

a0 = model.intercept_
a1 = model.coef_[0]

print(a0)
print(a1)
```

```
83.26735367241096
0.11737508839231389
```

In [ ]:
```
### (j) Compute Metrics ¤2, MSE, MAE, RMSE.
```

In [98]:
```
# Predicted values
y_pred = []
for i in range(n):
    y_val = a0 + a1 * x[i]
    y_pred.append(y_val)

# Initialize
SSE = 0
SAE = 0   # sum absolute error
mean_y = sum(y) / n
SST = 0   # total variance

for i in range(n):
    error = y[i] - y_pred[i]

    SSE += error ** 2
    SAE += abs(error)
    SST += (y[i] - mean_y) ** 2

# Metrics
MSE = SSE / n
RMSE = MSE ** 0.5
MAE = SAE / n
R2 = 1 - (SSE / SST)
```

```
# Output
# print("MSE:", round(float(MSE), 2))
# print("RMSE:", round(float(RMSE), 2))
# print("MAE:", round(float(MAE), 2))
# print("R^2:", round(float(R2), 3))

print("MSE:", round(MSE.item(), 2))
print("RMSE:", round(RMSE.item(), 2))
print("MAE:", round(MAE.item(), 2))
print("R^2:", round(R2.item(), 3))
```

```
MSE: 1192.4
RMSE: 34.53
MAE: 25.72
R^2: 0.632
```

In [ ]:

Dataset:-2.

(2)Consider the dataset for the simple linear regression.

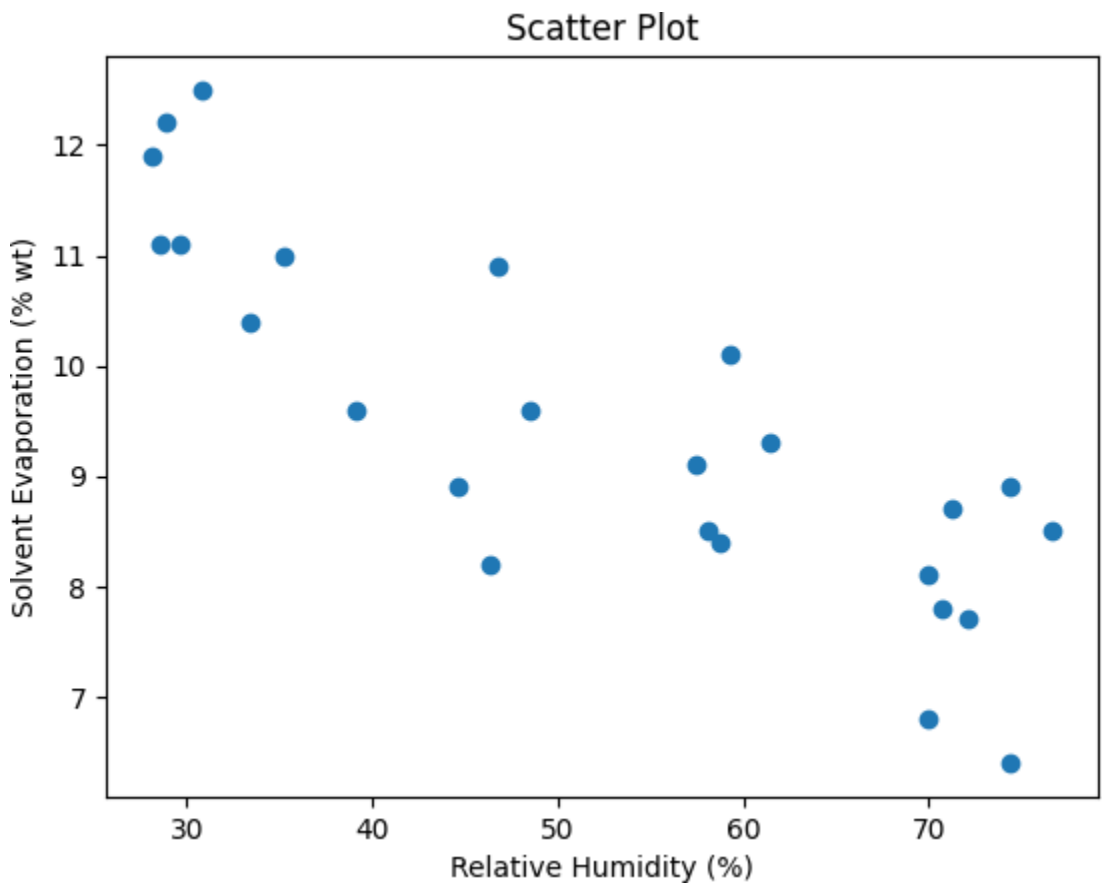
x (Relative Humidity %)	y (Solvent Evaporation % wt)
35.3	11.0
29.7	11.1
30.8	12.5
58.8	8.4
61.4	9.3
71.3	8.7
74.4	6.4
76.7	8.5
70.7	7.8
57.5	9.1
46.4	8.2
28.9	12.2
28.1	11.9
39.1	9.6
46.8	10.9
48.5	9.6
59.3	10.1
70.0	8.1
70.0	6.8
74.4	8.9
72.1	7.7
58.1	8.5
44.6	8.9
33.4	10.4
28.6	11.1

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.array([35.3,29.7,30.8,58.8,61.4,71.3,74.4,76.7,70.7,57.5,
              46.4,28.9,28.1,39.1,46.8,48.5,59.3,70.0,70.0,74.4,
              72.1,58.1,44.6,33.4,28.6])

y = np.array([11.0,11.1,12.5,8.4,9.3,8.7,6.4,8.5,7.8,9.1,
              8.2,12.2,11.9,9.6,10.9,9.6,10.1,8.1,6.8,8.9,
              7.7,8.5,8.9,10.4,11.1])

n = len(x)
```

In [4]:

```
plt.scatter(x, y)
plt.xlabel("Relative Humidity (%)")
plt.ylabel("Solvent Evaporation (% wt)")
plt.title("Scatter Plot")
plt.show()
```

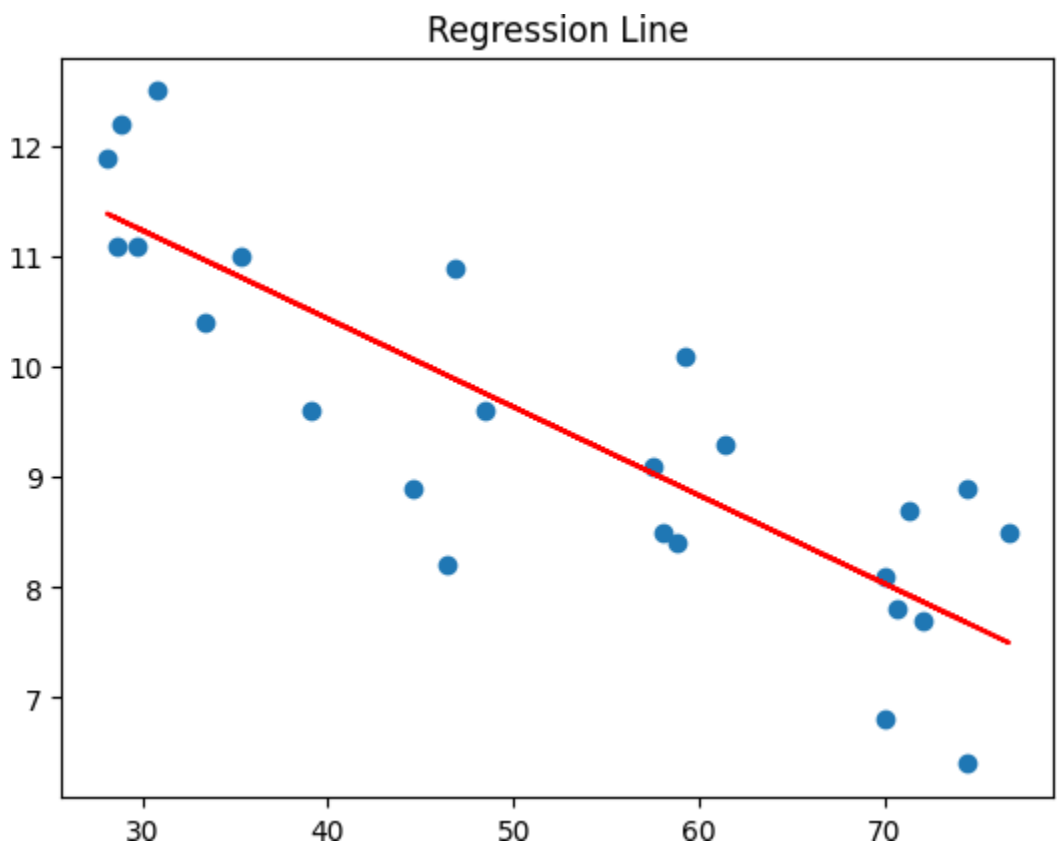


In [5]:

```
x_mean = np.mean(x)
y_mean = np.mean(y)

a1 = np.sum((x - x_mean)*(y - y_mean)) / np.sum((x - x_mean)**2)
a0 = y_mean - a1 * x_mean

print("Slope (a1):", a1)
print("Intercept (a0):", a0)
```

```
Slope (a1): -0.08006059146778506
Intercept (a0): 13.63886686883962
```

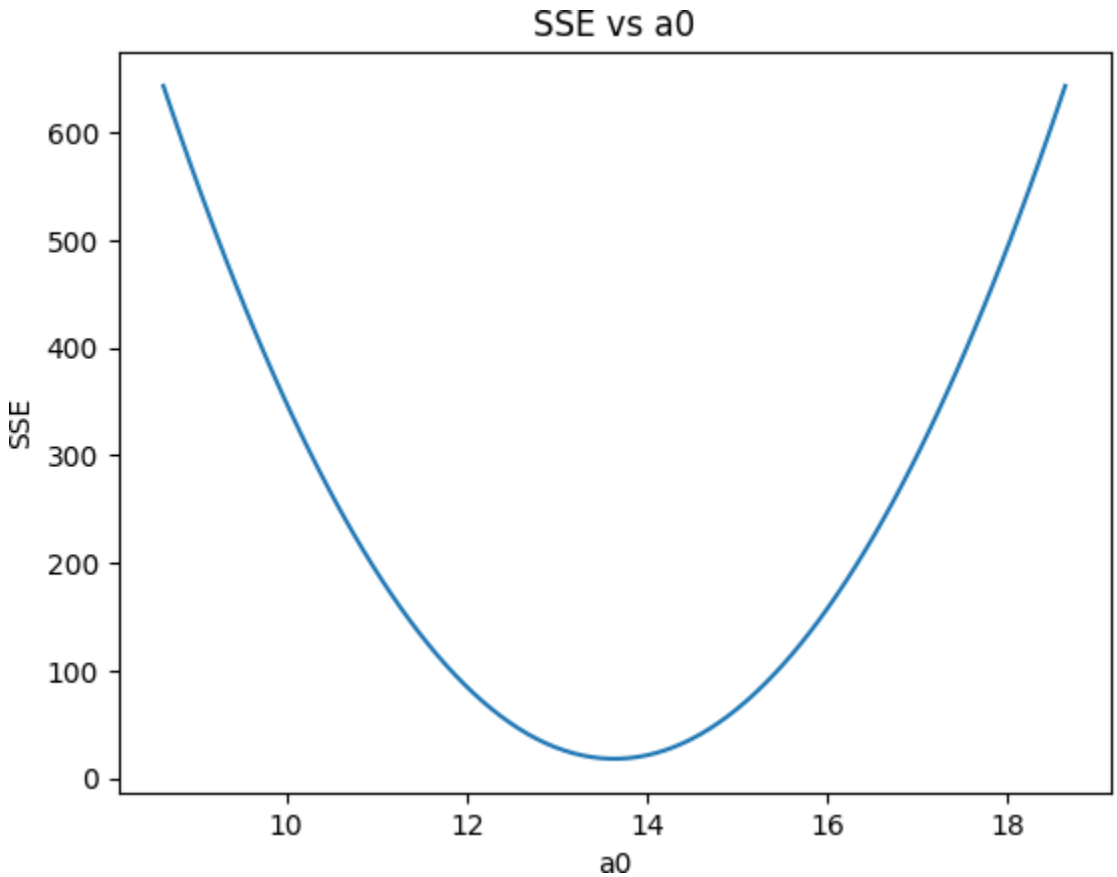
In [6]:
```
y_pred = a0 + a1 * x

plt.scatter(x, y)
plt.plot(x, y_pred, color='red')
plt.title("Regression Line")
plt.show()
```



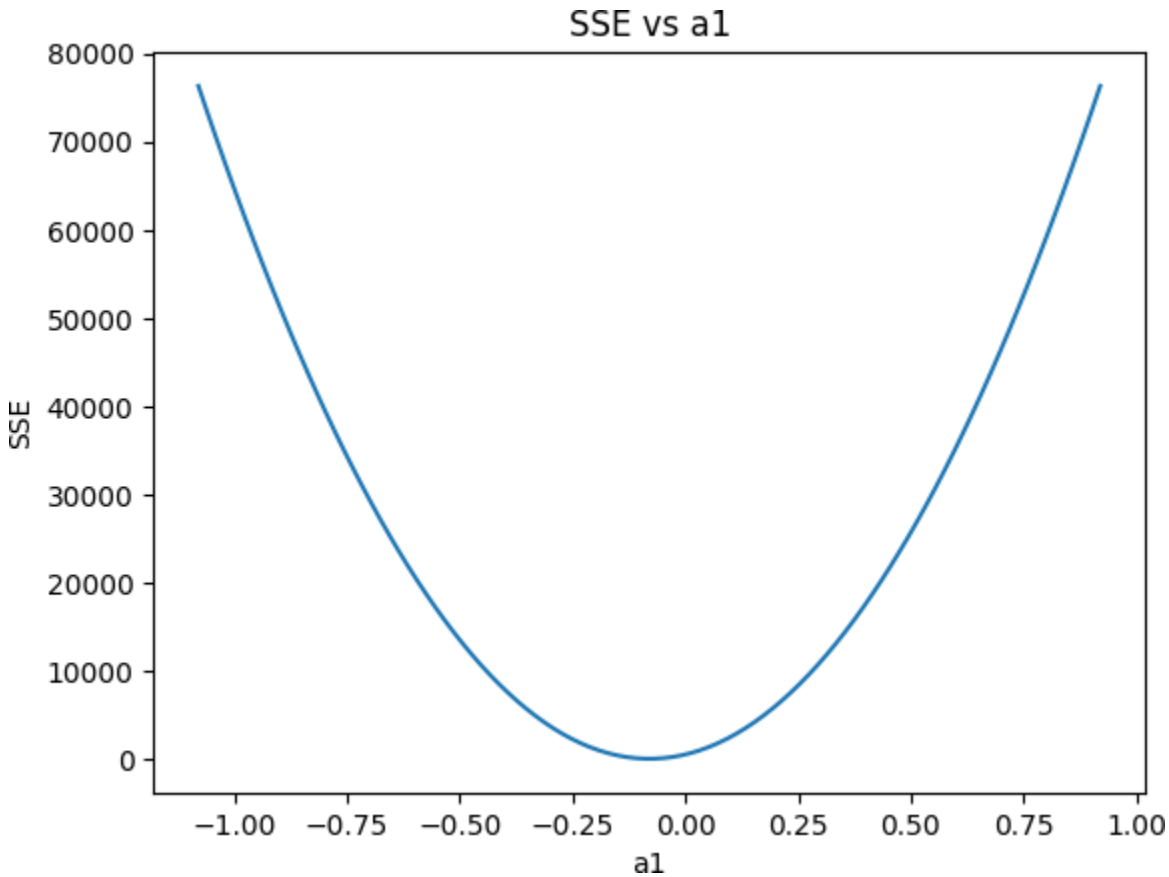
In [7]:
```
SSE = np.sum((y - y_pred)**2)
print("SSE:", SSE)
```

```
SSE: 18.060739189837225
```

In [8]:
```
a0_values = np.linspace(a0-5, a0+5, 100)
sse_a0 = []

for val in a0_values:
    y_temp = val + a1 * x
    sse_a0.append(np.sum((y - y_temp)**2))

plt.plot(a0_values, sse_a0)
plt.xlabel("a0")
plt.ylabel("SSE")
plt.title("SSE vs a0")
plt.show()
```

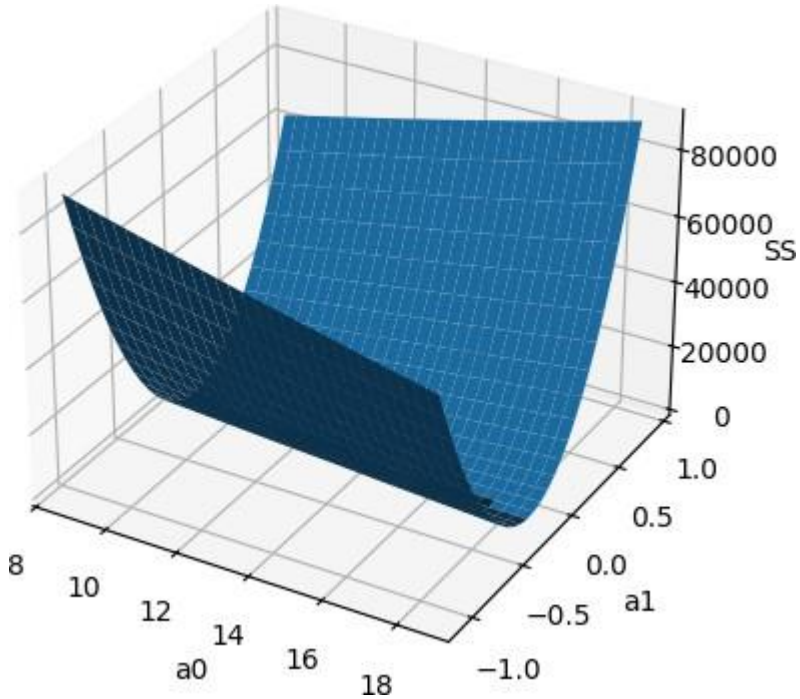


In [9]:
```
a1_values = np.linspace(a1-1, a1+1, 100)
sse_a1 = []

for val in a1_values:
    y_temp = a0 + val * x
    sse_a1.append(np.sum((y - y_temp)**2))

plt.plot(a1_values, sse_a1)
plt.xlabel("a1")
plt.ylabel("SSE")
plt.title("SSE vs a1")
plt.show()
```



In [10]:

```
from mpl_toolkits.mplot3d import Axes3D

a0_vals = np.linspace(a0-5, a0+5, 30)
a1_vals = np.linspace(a1-1, a1+1, 30)

A0, A1 = np.meshgrid(a0_vals, a1_vals)
S = np.zeros_like(A0)

for i in range(len(a0_vals)):
    for j in range(len(a1_vals)):
        y_temp = A0[j,i] + A1[j,i]*x
        S[j,i] = np.sum((y - y_temp)**2)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(A0, A1, S)
ax.set_xlabel('a0')
ax.set_ylabel('a1')
ax.set_zlabel('SSE')
plt.show()
```



In [11]:
```
a0_gd = 0
a1_gd = 0
lr = 0.0001
epochs = 1000

for i in range(epochs):
    y_pred = a0_gd + a1_gd * x

    da0 = (-2/n) * np.sum(y - y_pred)
    da1 = (-2/n) * np.sum(x * (y - y_pred))

    a0_gd -= lr * da0
    a1_gd -= lr * da1

print("GD a0:", a0_gd)
print("GD a1:", a1_gd)
```

```
GD a0: 0.2557581145008114
GD a1: 0.1505556942195959
```

In [12]:
```
X = np.column_stack((np.ones(n), x))
Y = y.reshape(-1,1)

b = np.linalg.inv(X.T @ X) @ X.T @ Y

print("Matrix a0:", b[0][0])
print("Matrix a1:", b[1][0])
```

```
Matrix a0: 13.638866868839587
Matrix a1: -0.08006059146778409
```

In [13]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(x.reshape(-1,1), y)

print("a0:", model.intercept_)
print("a1:", model.coef_[0])
```

```
Sklearn a0: 13.63886686883962
Sklearn a1: -0.08006059146778506
```

In [15]:
```
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

y_pred_lib = model.predict(x.reshape(-1,1))

R2 = r2_score(y, y_pred_lib)
MSE = mean_squared_error(y, y_pred_lib)
MAE = mean_absolute_error(y, y_pred_lib)
RMSE = np.sqrt(MSE)

print("R2:", R2)
print("MSE:", MSE)
print("MAE:", MAE)
print("RMSE:", RMSE)
```

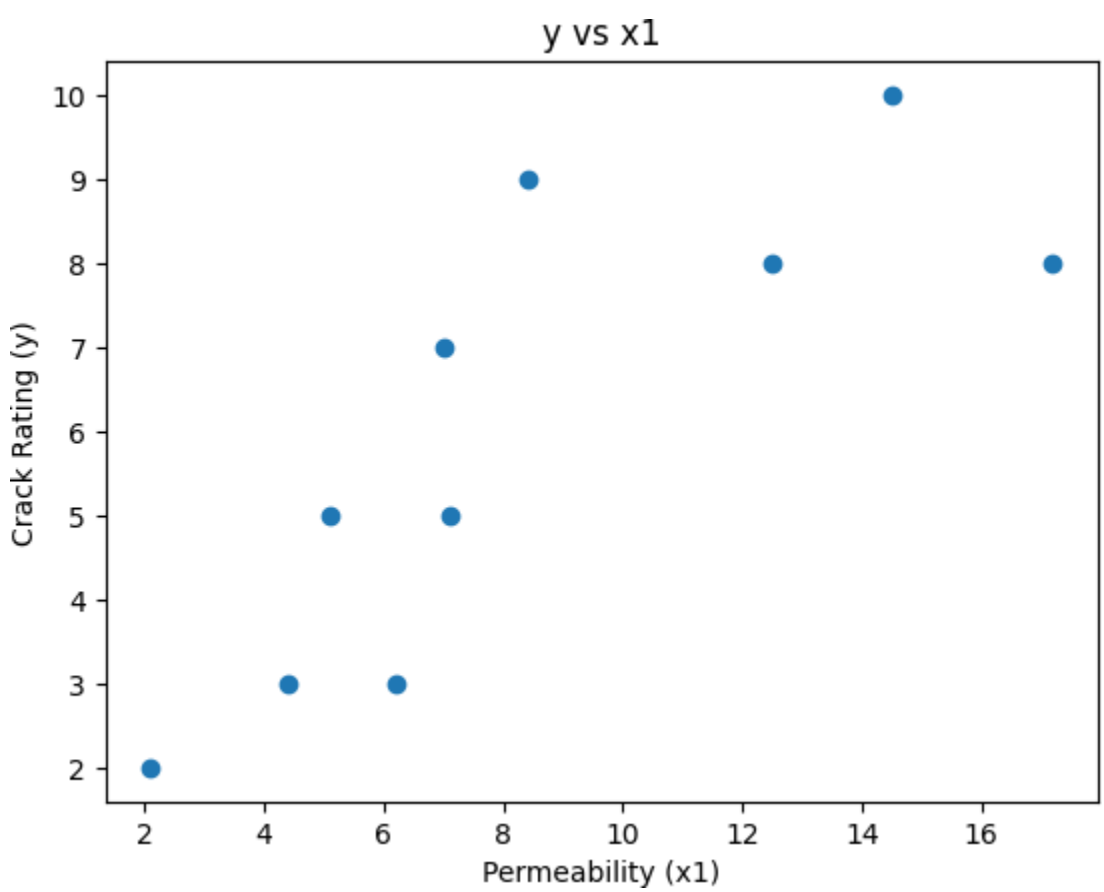
```
R2: 0.7173168552734491
MSE: 0.722429567593489
MAE: 0.7049925127232683
RMSE: 0.8499585681628775
```

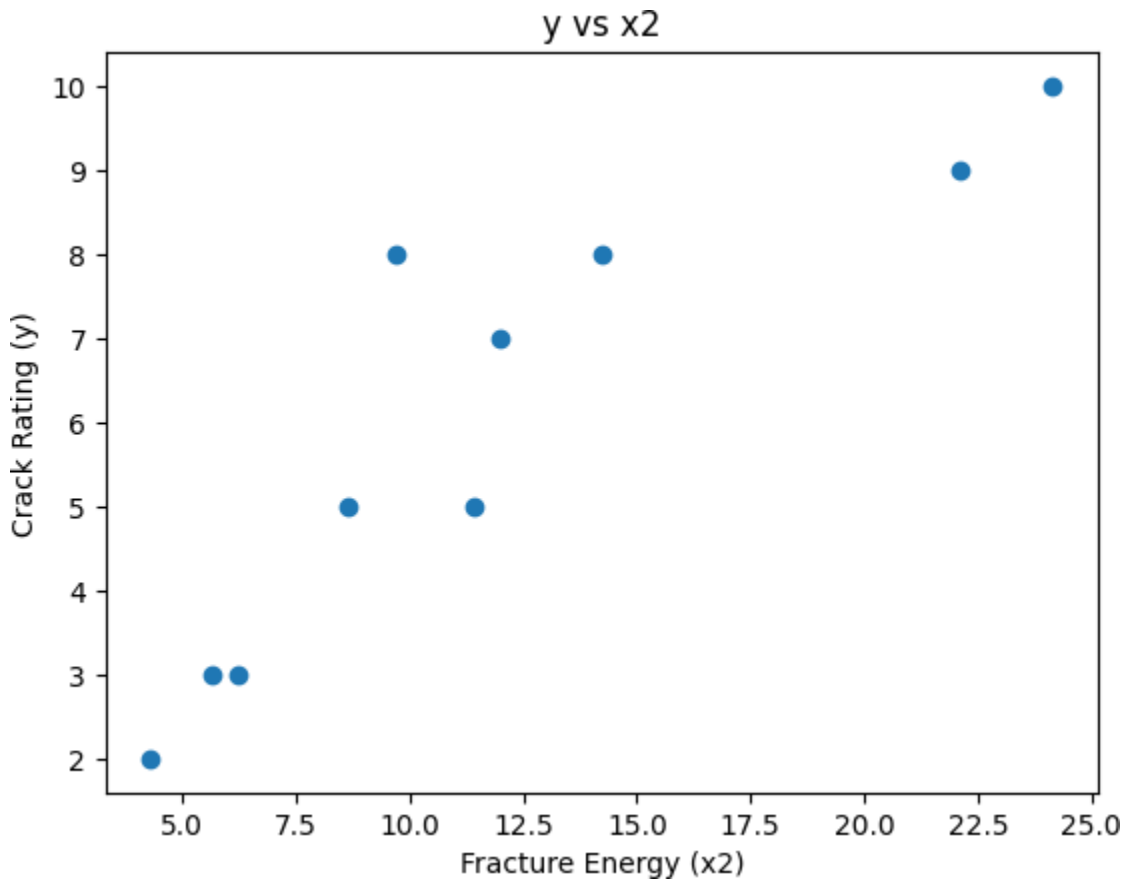
In [ ]:

Dataset:-3.
Q.3

In [9]:
```
import numpy as np
import matplotlib.pyplot as plt

# Data
y = np.array([2,9,5,10,3,3,8,7,8,5])

x1  =  np.array([2.1,8.4,5.1,14.5,4.4,6.2,12.5,7.0,17.2,7.1])
x2  =  np.array([4.31,22.11,11.40,24.15,6.21,5.65,9.71,12.00,14.25,8.63])

n = len(y)
```

In [10]:
```
plt.scatter(x1, y)
plt.xlabel("Permeability (x1)")
plt.ylabel("Crack Rating (y)")
plt.title("y vs x1")
plt.show()

plt.scatter(x2, y)
plt.xlabel("Fracture Energy (x2)")
plt.ylabel("Crack Rating (y)")
plt.title("y vs x2")
plt.show()
```





In [11]:
```
# Design matrix
X = np.column_stack((np.ones(n), x1, x2))
Y = y.reshape(-1,1)

# Formula
b = np.linalg.inv(X.T @ X) @ X.T @ Y

a0 = b[0][0]
a1 = b[1][0]
a2 = b[2][0]

print("a0 (Intercept):", a0)
print("a1 (x1 coeff):", a1)
print("a2 (x2 coeff):", a2)
```

```
a0 (Intercept): 0.8046744972685409
a1 (x1 coeff): 0.24521108935916924
a2 (x2 coeff): 0.26374698510779215
```

In [12]:
```
#  matrix
X = np.column_stack((np.ones(n), x1, x2))
Y = y.reshape(-1,1)

# Formula
b = np.linalg.inv(X.T @ X) @ X.T @ Y
```

```
a0 = b[0][0]
a1 = b[1][0]
a2 = b[2][0]

print("a0 (Intercept):", a0)
print("a1 (x1 coeff):", a1)
print("a2 (x2 coeff):", a2)
```

```
a0 (Intercept): 0.8046744972685409
a1 (x1 coeff): 0.24521108935916924
a2 (x2 coeff): 0.26374698510779215
```

In [13]:
```
y_pred = a0 + a1*x1 + a2*x2
print(y_pred)
```

```
[ 2.45636729  8.69589349  5.06196668 10.72972498  3.52147207  3.81515372
  6.43079634  5.68611594  8.78069977  4.82180971]
```

In [14]:
```
SSE = np.sum((y - y_pred)**2)
print("SSE:", SSE)
```

```
SSE: 6.603434348120018
```

In [15]:
```
a0_gd = 0
a1_gd = 0
a2_gd = 0

lr = 0.0001
epochs = 1000

for i in range(epochs):
    y_pred = a0_gd + a1_gd*x1 + a2_gd*x2

    da0 = (-2/n) * np.sum(y - y_pred)
    da1 = (-2/n) * np.sum(x1 * (y - y_pred))
    da2 = (-2/n) * np.sum(x2 * (y - y_pred))

    a0_gd -= lr * da0
    a1_gd -= lr * da1
    a2_gd -= lr * da2

print("GD a0:", a0_gd)
print("GD a1:", a1_gd)
print("GD a2:", a2_gd)
```

```
GD a0: 0.05223064328222176
GD a1: 0.27481813300899727
GD a2: 0.2940675547766966
```

In [10]:
```
from sklearn.linear_model import LinearRegression

X_lib = np.column_stack((x1, x2))

model = LinearRegression()
model.fit(X_lib, y)
```

```
print(" a0:", model.intercept_)
print(" a1, a2:", model.coef_)
```

```
a0: 0.8046744972685511
a1, a2: [0.24521109 0.26374699]
```

In [11]:

```
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

y_pred_lib = model.predict(X_lib)

R2 = r2_score(y, y_pred_lib)
MSE = mean_squared_error(y, y_pred_lib)
MAE = mean_absolute_error(y, y_pred_lib)
RMSE = np.sqrt(MSE)

print("R2:", R2)
print("MSE:", MSE)
print("MAE:", MAE)
print("RMSE:", RMSE)
```

```
R2: 0.9056652235982855
MSE: 0.6603434348120016
MAE: 0.6730769028902395
RMSE: 0.8126151824892281
```

In [ ]: